

CHALLENGE BRIEF

AI TRAINING HACKATHON

EVENT PORTAL • JUL 2026

PARTICIPANT

- 01 DASHBOARD
- 02 AGENDA
- 03 REFERENCE GUIDES
- 04 LAB MATERIALS
- 05 DAY 1 SUBMISSION
- 06 HACKATHON DOCS
- 07 TEAMS & SUBMISSIONS
- 08 THE LEADERBOARD

COGNITIVO HACKATHON

Build and fine-tune an evidence-grounded market signal agent over RBA, ASX, and AFR data.

CONTENTS

- [Challenge Scope](#)
- [Objective](#)
- [Required Model Roles](#)
- [Task Format](#)
- [Required Response](#)
- [Scoring](#)
- [Required Deliverables](#)
- [Rules and Constraints](#)
- [What a Good Answer vs a Bad Answer Looks Like](#)

CHALLENGE SCOPE

This hackathon evaluates how effectively an agent answers financial-market questions using the approved local datasets.

AREA	SCOPE
Question coverage	Easy, medium, and hard questions using one or more approved datasets.
Approved data	RBA cash-rate decisions, ASX company prices, and the AFR news corpus.
Evaluation split	15 public practice questions with answers; the remaining official questions stay with the organizers.
Supplied reasoning	Qwen3.6-35B-A3B-FP8 (called Qwen below) through the LiteLLM <code>agent-brain</code>

[● DAY]

@rpereirapersonal

[SIGN OUT]

AREA	SCOPE
brain	alias. Qwen performs planning, tool selection, tool-call generation, and iterative reasoning.
Fine-tuning target	<code>Llama-3.1-Nemotron-Nano-8B-v1</code> . Teams fine-tune Nemotron for grounded financial-domain answer synthesis.
Official score	30% fine-tuned model quality, 30% architecture and repository quality, and 40% hidden-question performance.

OBJECTIVE

Build and fine-tune a domain model, integrate it into a well-engineered agent, and demonstrate that the complete system can answer unseen financial-market questions. The benchmark includes easy, medium, and hard questions as well as single-dataset and cross-dataset questions.

The solution is expected to do more than produce a plausible sentence. It must show measurable fine-tuning quality, a reproducible repository and system architecture, grounded use of the supplied data, and clear answers on the hidden evaluation set.

REQUIRED MODEL ROLES

The submitted solution uses two model roles. Do not train Nemotron to replace the supplied Qwen reasoning brain or use Nemotron as the primary tool-calling model.

COMPONENT	REQUIRED RESPONSIBILITY
Qwen through <code>agent-brain</code>	Receives the question, plans the approach, selects <code>query_data</code> or retrieval tools, emits tool calls and arguments, reviews tool results, and decides whether another tool call is required. Participants do not fine-tune Qwen.
Agent runtime	Validates and executes Qwen's tool calls against the approved local datasets, records the trace, and returns structured results to Qwen.

COMPONENT	REQUIRED RESPONSIBILITY
	The model requests calls; the application code executes them.
Fine-tuned Nemotron through <code>DOMAIN_FT_MODEL</code>	Receives the question and accumulated verified tool results after the Qwen reasoning loop, then synthesizes the final concise financial-domain answer. This is the model participants fine-tune and assess against the supplied base Nemotron.

question

→ Qwen agent-brain plans and emits tool calls

→ agent runtime executes query_data / retrieval

→ tool results return to Qwen until reasoning

→ fine-tuned Nemotron synthesizes the final a

→ POST /query returns {"answer": "..."}

The organizer may also use Qwen as the independent LLM judge. That evaluation call is separate from the Qwen brain inside the submitted agent and does not replace the required fine-tuned Nemotron synthesis step.

The cluster bootstrap begins with `DOMAIN_PREDICT_MODE=mock` for pre-training integration tests. Teams must switch to `DOMAIN_PREDICT_MODE=llm` after serving their adapter and before official evaluation so the submitted solution actually uses the fine-tuned Nemotron model.

TASK FORMAT

For each question, the evaluator sends one JSON object in the format shown by `questions_template.json`: a single `question` field containing the question text. The agent must return one JSON object in the format shown by `answer_template.json`. The machine-readable rules in `validate.json` are used to confirm that the response can be parsed.

Questions may require direct retrieval, filtering, counting, financial calculations, chronological comparison, ranking, or reasoning across multiple datasets.

`public_questions.jsonl` contains 15 calibration cases in the same format the evaluation harness uses. Each line is a JSON object with `id`, `prompt`, `difficulty`, `datasets`, and a `grading` object listing the expected facts and their point values. Pass only the `prompt` field as the `question` to your agent. The `grading.components[].expected_fact` values show what the judge checks for.

Public questions are calibration cases. Use them to test retrieval, calculations, formatting, and the complete agent pipeline. Do not implement question-ID-specific hard-coded answers.

REQUIRED RESPONSE

Every response must include:

- `answer`: a direct response containing every requested component. This is the only response field scored in the hidden-question benchmark.
- `steps`: the number of reasoning steps the agent took. This optional integer is recorded for private organizer diagnostics.
- `tool_trace`: an ordered list of tool calls made during reasoning. This optional list is recorded for private organizer diagnostics.

Only `answer` is required. `steps` and `tool_trace` are optional but strongly recommended because they help organizers diagnose agent behavior. The response is validated against `validate.json`.

```
{
  "answer": "Direct answer with all requested va
  "steps": 3,
  "tool_trace": [
    {
      "tool": "tool_name",
      "args": {"param": "value"},
      "result": "tool output summary"
    }
  ]
}
```

SCORING

The final hackathon score is calculated from three independently assessed categories. Each category is normalized to a score out of 100 before its official weighting is applied.

CATEGORY	WEIGHT	WHAT IS ASSESSED
Fine-tuned model quality	30%	Quality and relevance of the fine-tuned model; measurable improvement over the supplied base model; training-data preparation; evaluation evidence; model behavior, robustness, and successful use of the fine-tuned model during inference.
Architecture and repository quality	30%	Agent and model architecture; appropriate use of deterministic tools, retrieval, and data processing; code quality and reliability; API-contract compliance; reproducibility; repository structure; README and run instructions; training artifacts; logs; pinned commit; security and documented limitations.
Hidden-question evaluation	40%	Performance of the submitted agent on unseen easy, medium, hard, single-dataset, and cross-dataset questions. Answers are graded using component-based correctness and partial credit.

```
final_score =  
    (fine_tuned_model_score * 0.30)  
    + (architecture_repository_score * 0.30)  
    + (hidden_question_score * 0.40)
```



FINE-TUNED MODEL QUALITY - 30%

Judges review the submitted training evidence and may test the declared fine-tuned model endpoint. Teams must

demonstrate that the fine-tuned model is genuinely used by the submitted solution. The assessment considers:

- The relevance, quality, and documented preparation of the fine-tuning data.
- The training method, configuration, hyperparameters, checkpoints, and model-selection rationale.
- Quantitative and qualitative comparison with the supplied base model on held-out or validation examples.
- Robustness, consistency, domain understanding, and avoidance of unsupported claims.
- Evidence that the final agent uses Qwen for planning and tool-call generation, then routes the verified tool results through the fine-tuned Nemotron model for final synthesis.

Training evidence must be reproducible and must not contain hidden evaluation data.

ARCHITECTURE AND REPOSITORY QUALITY - 30%

Judges inspect the public GitHub repository at the exact commit SHA declared in `submission.json`. The assessment considers:

- A clear end-to-end architecture covering Qwen planning/tool-call generation, runtime tool execution, fine-tuned Nemotron synthesis, retrieval, and data flow.
- Correct separation of responsibilities: Qwen selects and requests tools, application code executes them, and fine-tuned Nemotron writes the grounded final answer.
- Correct use of structured parsing and deterministic calculations for dataset-derived facts.
- Maintainable source code, sensible module boundaries, error handling, timeouts, and safe fallbacks.
- Compliance with `GET /health` and `POST /query`, including valid JSON responses.
- Complete README, run instructions, architecture explanation, training summary, and known limitations.
- Useful training artifacts, configurations, metrics, and non-sensitive logs in the required folders.

- A clean, accessible repository with no credentials, hidden evaluation material, or machine-specific secrets.

HIDDEN-QUESTION EVALUATION - 40%

Each hidden response is worth a maximum of 10 points. Points are allocated across the factual or analytical components explicitly requested by that question.

SCORING RULE	WHAT IS ASSESSED
Component-based correctness	Correct facts, dates, counts, calculations, rankings, sentiment, market direction, and every other output explicitly requested in the question.
Partial credit	Multi-part questions award the points attached to each independently correct requested component; one incorrect component does not erase correct components.
Equivalent expression	Equivalent date formats, harmless numeric formatting differences, and sentiment synonyms that preserve the reference meaning are accepted. Each case's <code>grading.tolerance_note</code> defines any permitted numeric tolerance or rounding.
Response validity	Only <code>answer</code> is required for automated hidden-question scoring. <code>steps</code> and <code>tool_trace</code> are retained for private diagnostics. Malformed or missing <code>answer</code> fields score zero.

The grader requires only information requested by the prompt. Extra dates, prices, quotations, or article drivers appearing in supporting material are not mandatory unless the question asks for them.

Response-Time Rules

RESPONSE TIME	HIDDEN-QUESTION SCORING EFFECT
60 seconds or less	Full earned points.

RESPONSE TIME	HIDDEN-QUESTION SCORING EFFECT
More than 60 seconds and no more than 300 seconds	20% is deducted from the points earned for that question.
More than 300 seconds	Timeout and zero points for that question.

`GET /health` is a hard gate for the hidden-question evaluation. If the registered agent does not return HTTP 200 during the pre-evaluation health check, the team is skipped and receives no hidden-question points for that run.

The harness sends up to three hidden questions concurrently to each team by default. The submitted agent, tool runtime, and model servers must safely handle at least three simultaneous `/query` requests without mixing state or responses.

RANKING AND FEEDBACK

The final ranking uses the weighted score across all three categories. The public leaderboard shows rank, team, and final score. Organizers retain private diagnostics, and each team may receive a sanitized report for its own submission. Hidden expected facts and other teams' private data are not shared.

REQUIRED DELIVERABLES

- A working agent that accepts the specified question object and returns the specified response object.
- Source code and any data-query or retrieval tools created by the team.
- Fine-tuning or training evidence: scripts, preparation notes, configuration, logs, and a short model summary.
- Evidence comparing the supplied base model with the team's fine-tuned model.
- A reachable fine-tuned model or a documented organizer-approved method for technical model assessment.

- A final `submission.json` registering the team, repository commit, and agent endpoint.
- A valid sample `Participant_Package/answer_template.json` demonstrating the agent's per-question response contract.
- A README explaining the architecture, how to run the agent, and known limitations.
- Useful logs or traces that allow the organizers to diagnose failed requests.

Submit as a `TeamSubmission/` repository containing `README.md`, `submission.json`, `src/`, `training/`, `logs/`, and `Participant_Package/`. See [Submission Guide → Required Repository Structure](#) for the exact layout and a file-by-file breakdown of what each path must contain.

RULES AND CONSTRAINTS

- Use only approved local datasets and services during official scoring.
- Do not use unrestricted external browsing during scoring.
- Do not alter the source datasets.
- All official responses must be valid JSON and follow the required contract.
- Return a response for every question, even when evidence is insufficient. State the limitation clearly in the `answer` field instead of returning an empty response or inventing a figure.
- Malformed, crashing, or timed-out responses may receive no credit for that case.
- Keep secrets and credentials out of submitted files and logs.

TECHNICAL REFERENCE

Dataset schemas, AFR search rules, the fine-tuning baseline configuration, and model-serving endpoints are documented in [Setup Instructions](#). Those points are non-negotiable for reproducibility: scores are computed by running the same tool calls against the same data, so a

different search scope or field set will not match the reference answers.

WHAT A GOOD ANSWER VS A BAD ANSWER LOOKS LIKE

SLOW-RESPONSE PENALTY, WORKED EXAMPLE

Applying the Response-Time Rules above: your answer earns 8/10, but took 83 seconds. $\text{Penalty} = 8 \times 0.20 = 1.6$ pts deducted. $\text{Final} = 6.4/10$.

Design implication: Aim for ≤ 3 tool calls per question. If your agent loops more than 5 times or calls `list` on a large dataset, it will likely exceed 60 seconds on complex questions.

WHAT ZERO LOOKS LIKE — AND WHY

Example 1: No tool use (base model, first run)

Question: From the first RBA record to the last, how many cash-rate decisions changed the rate, and how many were increases versus decreases?

Answer (base model, 0% score):

"The user is asking for statistics about RBA cash rate decisions from the first record to the last. I need to find the RBA records in the team's corpora. I will use the `retrieve` tool to search for 'RBA cash rate decisions' or 'RBA board minutes' or similar. Then I will analyze the retrieved documents to count the total number of decisions that changed the rate, and how many were increases vs decreases..."

Why it fails:

- A retrieval tool over news articles will never return structured RBA decision counts.
- The answer never produces a number.
- This is the agent "thinking out loud" rather than acting — a properly fine-tuned domain model eliminates this.

Example 2: Tool called but wrong metric

Question: What was the longest period the RBA held rates unchanged?

Answer (early iteration, 0% score):

"Based on my search of the RBA data, the cash rate was held steady for an extended period during 2016-2019."

Why it fails:

- The agent didn't call the correct structured-data metric for the longest hold streak.
- It likely listed raw rows and tried to eyeball the output instead of computing the answer.
- The answer gives no exact count of days, no start/end dates — zero components verified.

Example 3: Partial credit — right number, missing context

Question: What is the highest cash-rate target ever in the RBA dataset, and how many records show it?

Answer (partial, 50% score):

"The highest cash-rate target in the RBA dataset is 4.75, which first appeared on the effective date 2010-11-03. This maximum rate appears across 11 effective-date records."

Why it only gets 50%: The grading had 3 components: the rate (4.75 ✓), the first effective date (2010-11-03 X — the judge expected 2010-11-02), and the record count (11 ✓). Two of three components matched. **Date precision matters.**

WHAT FULL MARKS LOOKS LIKE

Easy question — 100% score

Question: What is the lowest cash-rate target in the RBA dataset, when did it first take effect, and how many decision records show that rate?

Answer:

"The lowest cash-rate target in the RBA dataset was 0.1, which first took effect on 2020-11-04, and 16 decision records show that rate."

Why it works:

- One sentence, three facts, no hedging.
- All three grading components satisfied: 0.1, 2020-11-04, 16.

- Concise — the judge doesn't need to find the answer buried in paragraphs.

Medium question — 100% score

Question: What was the longest stretch between two non-zero RBA rate changes?

Answer:

"The longest stretch between two non-zero RBA rate changes was 1036 days, lasting from 2016-08-03 to 2019-06-05, during which the rate held at 1.5 before changing to 1.25."

Why it works:

- The agent retrieved the exact structured result rather than estimating it.
- Every component is stated: duration (1036 days), start date (2016-08-03), end date (2019-06-05), rate during hold (1.5), rate after (1.25).
- No invented context, no hedging.

Hard question — 100% score

Question: How many hikes occurred during the 2022-2023 tightening cycle, and what was the cumulative increase?

Answer:

"There were 13 hikes during the 4 May 2022 to 8 Nov 2023 tightening cycle, resulting in a cumulative increase of 4.25 percentage points. The target rate immediately before the first hike was 0.1 percent, and the final target reached on 8 Nov 2023 was 4.35 percent."